Facultade de Informática
UNIVERSIDADE DA CORUÑA

TRABALLO FIN DE GRAO
GRAO EN ENXEÑARÍA INFORMÁTICA
MENCIÓN EN ENXEÑARÍA DE COMPUTADORES

Sistema de captura y procesado de datos IMU en tiempo real para estimación de movimiento vertical

Estudante: Mateo Rivela Santos

Dirección: Dirección por asignar

A Coruña, marzo de 2026.

A mi familia, por su apoyo constante en cada etapa de este camino, y a todas las personas que me animaron a seguir aprendiendo cuando el resultado no salía a la primera.

Agradecimientos

Agradezco a mi entorno personal por la paciencia y el apoyo durante el desarrollo de este trabajo. Agradezco también al profesorado de la FIC por la base técnica y metodológica que ha hecho posible este proyecto, y a las personas que me ayudaron con pruebas, validaciones y revisión de resultados.

Resumen

Este trabajo presenta el desarrollo de un producto software móvil para captura y procesado de datos inerciales, con Android como plataforma ubicua principal. El sistema se diseña con dos modos de operación: modo autónomo (captura y procesado en el propio teléfono) y modo pantalla (visualización de resultados procesados por una plataforma externa como ESP32+MPU6050).

La solución implementada integra captura de sensores, normalización temporal, cálculo de métricas de oleaje y visualización en una misma aplicación, además de exportación de sesiones con parámetros y resultados. El algoritmo de procesado se utiliza como componente técnico del producto, no como fin en sí mismo.

Los resultados obtenidos muestran que la arquitectura de plataforma es viable y extensible, y permiten definir una comparativa homogénea entre Android y ESP32 para evaluar rendimiento, robustez y limitaciones en escenarios equivalentes.

Abstract

Este traballo presenta o desenvolvemento dun produto software móbil para captura e procesado de datos inerciais, con Android como plataforma ubicua principal. O sistema deseñouse con dous modos de operación: modo autónomo (captura e procesado no propio teléfono) e modo pantalla (visualización de resultados procesados por unha plataforma externa como ESP32+MPU6050).

A solución integra captura de sensores, normalización temporal, cálculo de métricas de ondada, visualización e exportación de sesións con parámetros e resultados nunha mesma aplicación. O algoritmo de procesado emprégase como compoñente técnico do produto e non como obxectivo final por si mesmo.

Os resultados mostran que a arquitectura de plataforma é viable e extensible, e permiten definir unha comparativa homoxénea entre Android e ESP32 para avaliar rendemento, robustez e limitacións en escenarios equivalentes.

Palabras clave:

- Unidad de medida inercial (IMU)
- Android
- Plataforma ubicua
- Producto software
- Procesado on-device
- Arquitectura móvil
- Comparativa Android-ESP32

Keywords:

- Unidade de medida inercial (IMU)
- Android
- Plataforma ubicua
- Produto software
- Procesado on-device
- Arquitectura móbil
- Comparativa Android-ESP32

Índice general

1 Introducción **1**

1.1 Motivación del problema 1

1.2 Objetivos 2

1.2.1 Objetivo general 2

1.2.2 Objetivos específicos 2

1.3 Alcance y limitaciones 2

1.4 Metodología de desarrollo 3

1.5 Estructura de la memoria 4

2 Estado del arte y contexto tecnológico **5**

2.1 Medición de oleaje y parámetros de estado de mar 5

2.1.1 Parámetros característicos 5

2.2 Plataforma móvil como nodo ubicuo 6

2.2.1 Ventajas y limitaciones frente a un sistema dedicado 6

2.3 Procesado inercial para oleaje 6

2.3.1 Qué puede y qué no puede estimarse con una IMU 6

2.4 Métodos espectrales y estimación mediante Welch 7

2.5 Sistemas embebidos para adquisición inercial 7

2.5.1 Soluciones móviles frente a soluciones embebidas 7

2.6 Ventanas temporales y robustez 8

2.7 Comparativa de plataformas 8

2.8 Posicionamiento del trabajo 8

3 Análisis del problema, requisitos, planificación y presupuesto **10**

3.1 Descripción general del sistema a desarrollar 10

3.2 Requisitos funcionales 10

3.3 Requisitos no funcionales 11

3.4 Restricciones técnicas 11

3.5 Casos de uso principales 11
3.6 Enfoque de trabajo 12
3.7 Fases del proyecto 13
3.8 Plan de pruebas y control de riesgos 13
3.9 Presupuesto orientativo 13

4 Diseño de la plataforma 14
4.1 Enfoque metodológico de diseño 14
4.2 Arquitectura funcional 14
4.2.1 Arquitectura física 15
4.2.2 Arquitectura lógica y software 15
4.3 Diseño del bloque IMU 15
4.4 Diseño del subsistema de adquisición 16
4.4.1 Adquisición en modo móvil 16
4.4.2 Adquisición en modo embebido 16
4.4.3 Diseño del almacenamiento de sesiones 16
4.4.4 Ventanas temporales y duración de captura 16
4.5 Diseño del flujo Android 17
4.6 Flujo de datos del sistema 17
4.7 Diseño de comparativa entre plataformas 17
4.7.1 Modo móvil 17
4.7.2 Modo sistema embebido 17
4.8 Diseño del subsistema de procesado 17
4.8.1 Metodología de procesado espectral 20
4.8.2 Entrada y preprocesado 20
4.8.3 Bloque espectral 21
4.8.4 Cálculo de momentos espectrales 21
4.8.5 Obtención de parámetros de oleaje 22
4.8.6 Interpretación física de resultados 22
4.8.7 Salida de resultados 23
4.9 Diseño hardware del sistema embebido 23

5 Implementación del sistema 24
5.1 Captura y normalización 24
5.1.1 Implementación de la captura en Android 24
5.1.2 Implementación del formato de sesión 24
5.1.3 Decisiones de diseño del almacenamiento 24
5.2 Pipeline de procesado integrado 25

5.2.1 Estimación de frecuencia efectiva y calidad temporal 25
5.2.2 Obtención de la señal dinámica útil 26
5.2.3 Estimación de PSD mediante Welch 26
5.2.4 Implementación de modos de procesado 27
5.2.5 Cálculo de momentos espectrales 27
5.2.6 Trazabilidad del resultado 28
5.3 Visualización y exportación 28
5.4 Implementación de la aplicación Android 28
5.4.1 Modo móvil 28
5.4.2 Modo ESP32 28
5.4.3 Gestión de sesiones y visualización 29
5.4.4 Estructura funcional de la app 29
5.4.5 Decisiones de diseño software 29
5.5 Implementación de referencia para iteración técnica 29
5.6 Implementación del subsistema embebido 29
5.6.1 Firmware principal del ESP32 30
5.6.2 Firmware y scripts de diagnóstico 30
5.6.3 Estado actual y evolución prevista 30

6 Validación y evaluación 32
6.1 Metodología experimental 32
6.1.1 Escenarios de prueba 32
6.2 Estrategia temporal de evaluación 33
6.3 Configuración de las pruebas 33
6.3.1 Procedimiento de análisis 33
6.4 Métricas de evaluación 34
6.5 Plantilla de comparativa Android vs ESP32+MPU6050 34
6.6 Criterios de aceptación 34
6.7 Resultados 35
6.7.1 Resultados del modo móvil 35
6.7.2 Resultados del sistema embebido 35
6.7.3 Resultados del procesado espectral 36
6.7.4 Comparación móvil vs sistema embebido 36
6.7.5 Resultados de robustez e integración 36
6.8 Discusión de resultados 37
6.8.1 Análisis crítico de los resultados 37
6.8.2 Validez del enfoque basado en IMU 38
6.8.3 Ventajas e inconvenientes de cada plataforma 38

6.8.4 Limitaciones del trabajo 38
6.8.5 Lecciones aprendidas 38

7 Conclusiones **39**
7.1 Conclusiones principales 39
7.2 Conclusión metodológica clave (a remarcar en la defensa) 39
7.3 Implicaciones prácticas para el diseño experimental 40
7.4 Aportaciones 40
7.5 Trabajo futuro 41
7.5.1 Líneas futuras prioritarias 41

A Material adicional **43**
A.1 Scripts principales 43
A.2 Comandos de referencia 43
A.2.1 Captura 43
A.2.2 Procesado rápido sin gráfica 43
A.2.3 Procesado paso a paso con validación temporal 43
A.3 Datos de prueba 44

Bibliografía **46**

Índice de figuras

1.1 Esquema conceptual del problema abordado por el sistema. 2

2.1 Relación conceptual entre espectro de oleaje y parámetros característicos. . . . 6

3.1 Casos de uso principales del sistema. 12

4.1 Arquitectura funcional del TFG con Android como plataforma central. 15
4.2 Arquitectura física del sistema y conexiones entre subsistemas. 15
4.3 Flujo Android: captura, procesado on-device, visualización y exportación. . . . 18
4.4 Flujo de datos extremo a extremo del sistema. 19
4.5 Flujo de comparativa entre plataformas con métricas unificadas. 19
4.6 Bloque de entrada y preprocesado del pipeline. 21
4.7 Conversión desde espectro de elevación a parámetros de oleaje. 22
4.8 Esquema del subsistema embebido y conexiones principales. 23

5.1 Ejemplo de señal inercial antes y después del preprocesado. 26
5.2 Ejemplo de estimación espectral y conversión a elevación. 27
5.3 Flujo de navegación de la aplicación Android. 29
5.4 Visualización por etapas del pipeline utilizada en la fase de validación. 30
5.5 Estado actual del subsistema embebido durante desarrollo y diagnóstico. . . . 31

6.1 Protocolo experimental seguido en la validación. 33
6.2 Resultados representativos de una sesión en modo móvil. 35
6.3 Resultados representativos del modo embebido. 35
6.4 Ejemplo completo del procesado espectral sobre una sesión. 36
6.5 Comparación de resultados entre plataforma móvil y sistema embebido. 37

Índice de cuadros

2.1 Posicionamiento resumido del trabajo frente a soluciones previas. 8

6.1 Plantilla de comparación Android vs ESP32+MPU6050 con protocolo homogéneo. 34

Capítulo 1

Introducción

Este trabajo fin de grado se centra en una idea principal: usar el teléfono móvil como plataforma ubicua para capturar y procesar datos inerciales, y demostrar su viabilidad en un caso de uso de caracterización de oleaje. El valor del sistema no está en un sensor concreto, sino en el producto software: su arquitectura de plataforma, su operación en distintos modos y su capacidad de despliegue práctico.

1.1 Motivación del problema

La señal inercial medida por un dispositivo móvil combina aceleración útil, gravedad, ruido, deriva y cambios de orientación, lo que dificulta una interpretación directa [1]. Para transformar estas medidas en métricas útiles de estado de mar, se requiere un pipeline de procesado robusto y consistente.

La motivación de este TFG no se limita al desarrollo de una aplicación móvil. El problema real es construir un sistema accesible de adquisición, procesado y visualización capaz de caracterizar el estado del mar con componentes de bajo coste y alta disponibilidad. En este contexto, los sensores inerciales permiten explorar una alternativa ligera frente a instrumentación oceanográfica más compleja o costosa, siempre que el tratamiento de señal y la arquitectura global del sistema sean adecuados.

En este trabajo se adopta como método principal un enfoque tipo OpenMetBuoy: estimación espectral mediante Welch, conversión de espectro de aceleración a elevación y cálculo de métricas de oleaje (H_s, T_z, T_c). La implementación se orienta a ejecución *on-device* en Android, con una fase posterior de comparación frente a ESP32+MPU6050.

Figura pendiente

Esquema conceptual recomendado:
oleaje real → sensor IMU → adquisición
→ procesado espectral → parámetros estimados

Figura 1.1: Esquema conceptual del problema abordado por el sistema.

1.2 Objetivos

1.2.1 Objetivo general

Diseñar e implementar una plataforma móvil capaz de capturar y procesar datos inerciales para estimación de movimiento vertical de olas, con salida reproducible y comparable entre plataformas.

1.2.2 Objetivos específicos

- Implementar en Android un modo autónomo de captura y procesado local.
- Estandarizar formato de sesión y exportación (CSV/JSON) con metadatos de adquisición.
- Aplicar un pipeline espectral tipo OpenMetBuoy para estimar H_s, T_z y T_c.
- Incluir visualización de olas $h(t)$, métricas y estado de validez de ventana.
- Implementar un segundo modo Android para mostrar resultados procedentes de ESP32+MPU6050.
- Definir una comparativa experimental homogénea Android vs ESP32 con métricas comunes.

1.3 Alcance y limitaciones

El alcance de la memoria es la estimación de movimiento vertical relativo en superficie, no navegación inercial completa. No se aborda estimación absoluta de orientación/posición con fusión avanzada magnetómetro–GNSS, por su mayor complejidad [2].

Las limitaciones principales consideradas son:

- Sensibilidad de la estimación a orientación y calidad de medida.

- Necesidad de ventanas largas (10 a 20 min) para resultados espectrales robustos.
- Menor fiabilidad del modo rápido en ventanas cortas (30 s), útil solo como diagnóstico.
- Diferencias de tasa efectiva de muestreo entre plataformas.
- Estado todavía iterativo de la integración hardware en la rama embebida.

1.4 Metodología de desarrollo

La memoria se apoya en cuatro marcos metodológicos complementarios, cada uno orientado a una dimensión distinta del trabajo:

- **metodología de desarrollo incremental**, para construir el sistema por fases y reducir riesgo técnico;
- **enfoque de diseño modular de sistema ciberfísico**, para estructurar adquisición, comunicación, procesado y visualización como subsistemas coordinados;
- **metodología de procesado espectral basada en Welch**, como núcleo técnico del tratamiento de señal;
- **metodología experimental comparativa**, para evaluar Android y sistema embebido bajo un protocolo homogéneo.

En términos prácticos, la metodología de desarrollo aplicada es incremental y orientada a validación:

1. Definición de arquitectura funcional con Android como núcleo.
2. Implementación y pruebas del pipeline de procesado en entorno reproducible.
3. Validación de estabilidad del método con ventanas largas y análisis de límites en ventanas cortas.
4. Integración del modo Android display para consumir resultados externos.
5. Diseño del protocolo de comparación Android vs ESP32+MPU6050.

1.5 Estructura de la memoria

La memoria se organiza de la siguiente forma:

- En el Capítulo 2 se resume el estado del arte estrictamente necesario para justificar decisiones de producto.
- En el Capítulo 3 se describe la metodología de desarrollo, la planificación y el presupuesto de trabajo.
- En el Capítulo 4 se define la arquitectura funcional de la plataforma y sus módulos, incluyendo IMU y modos de operación.
- En el Capítulo 5 se documenta la implementación técnica del producto y el formato de datos.
- En el Capítulo 6 se presenta el plan de pruebas y la validación experimental Android vs ESP32+MPU6050.
- En el Capítulo 7 se recogen conclusiones, aportaciones y trabajo futuro.
- En el Apéndice A se incluye material complementario.

Capítulo 2

Estado del arte y contexto tecnológico

Este capítulo recoge únicamente el contexto técnico necesario para justificar la solución propuesta, manteniendo el foco en desarrollo de producto.

2.1 Medición de oleaje y parámetros de estado de mar

La caracterización del oleaje no se expresa normalmente mediante la altura instantánea de una ola concreta, sino mediante parámetros estadísticos del estado del mar calculados sobre una ventana temporal suficientemente larga. Entre los más utilizados destacan la altura significativa H_s, el período medio por cruces por cero T_z, el período de pico T_p y otros descriptores derivados del espectro de energía.

Desde el punto de vista ingenieril, esta distinción es importante porque condiciona tanto la estrategia de adquisición como el tipo de validación experimental. El objetivo del sistema desarrollado en este TFG es, por tanto, estimar parámetros representativos del estado del mar, no reconstruir con precisión absoluta la elevación instantánea en cada muestra.

2.1.1 Parámetros característicos

- H_s: altura significativa, vinculada a la energía total contenida en el espectro.
- T_z: período medio asociado a cruces por cero o a relaciones entre momentos espectrales.
- T_p: período correspondiente al pico dominante del espectro.
- T_c: período característico derivado de momentos de orden superior.

Estos parámetros son especialmente útiles porque resumen el estado del mar con magnitudes interpretables y comparables entre plataformas de medida heterogéneas.

Figura pendiente

Figura sugerida:
relación entre espectro de energía, momentos espectrales
y parámetros H_s, T_z, T_p, T_c

Figura 2.1: Relación conceptual entre espectro de oleaje y parámetros característicos.

2.2 Plataforma móvil como nodo ubicuo

El uso de smartphone como nodo de captura aporta disponibilidad, conectividad y capacidad de cómputo en un único dispositivo. Para este TFG, Android se adopta como plataforma principal por:

- acceso directo a sensores inerciales integrados;
- ecosistema de desarrollo maduro y despliegue rápido;
- capacidades locales de almacenamiento, visualización y exportación.

2.2.1 Ventajas y limitaciones frente a un sistema dedicado

El smartphone reduce barrera de entrada y acelera el desarrollo extremo a extremo. Sin embargo, introduce condicionantes como control limitado del hardware sensor, variabilidad entre modelos y menor trazabilidad física que un diseño embebido dedicado.

2.3 Procesado inercial para oleaje

La estimación de estado de mar a partir de IMU se aborda en la literatura con técnicas espectrales. En este trabajo se adopta un enfoque tipo OpenMetBuoy por su trazabilidad y aplicabilidad práctica, evitando desarrollar un algoritmo propio desde cero.

El objetivo del TFG no es investigar un nuevo método, sino integrar un método existente dentro de una plataforma usable y evaluable.

2.3.1 Qué puede y qué no puede estimarse con una IMU

Una IMU permite observar dinámica inercial y, mediante procesado adecuado, derivar parámetros estadísticos de oleaje. No obstante, la reconstrucción directa y precisa de altura

instantánea mediante doble integración presenta problemas bien conocidos de deriva y acumulación de error, por lo que la estrategia del trabajo se orienta a estimación espectral por ventana.

2.4 Métodos espectrales y estimación mediante Welch

Entre los métodos de estimación espectral, la técnica de Welch presenta un compromiso adecuado entre robustez, coste computacional y facilidad de integración en plataformas de recursos limitados. Su uso permite estimar la densidad espectral de potencia a partir de segmentos solapados y posteriormente calcular momentos espectrales a partir de los cuales se obtienen los parámetros de oleaje.

La relevancia de este enfoque para el proyecto es doble. Por una parte, proporciona una base técnica alineada con sistemas reales de observación del mar. Por otra, evita depender de doble integración directa de la aceleración en el dominio temporal, una operación muy sensible a deriva, orientación y ruido acumulado.

2.5 Sistemas embebidos para adquisición inercial

Además del uso de sensores integrados en el teléfono móvil, la literatura y la práctica industrial emplean soluciones embebidas con IMU dedicadas para separar la adquisición física del subsistema de visualización o supervisión. En este trabajo, la arquitectura embebida se plantea mediante un bloque MCU+ESP32+IMU, donde el ESP32 actúa como nodo de comunicación con la aplicación Android.

Esta aproximación permite estudiar el equilibrio entre dos estrategias de diseño:

- una solución integrada y ubicua basada en smartphone;
- una solución más dedicada y modular basada en electrónica embebida.

2.5.1 Soluciones móviles frente a soluciones embebidas

La comparación entre ambas arquitecturas no debe entenderse como una sustitución directa, sino como una evaluación de compromisos: ubicidad frente a control de hardware, simplicidad de despliegue frente a modularidad física, y rapidez de iteración frente a robustez instrumental.

2.6 Ventanas temporales y robustez

Una conclusión técnica recurrente en trabajos y sistemas similares es la necesidad de ventanas de adquisición suficientemente largas para estabilizar métricas espectrales. Por ello, este trabajo diferencia dos modos:

- modo robusto con ventanas de 10 a 20 minutos;
- modo rápido para diagnóstico local, no equivalente al resultado robusto.

2.7 Comparativa de plataformas

Además del modo Android autónomo, se plantea una segunda fase con ESP32+MPU6050 para contrastar viabilidad en hardware dedicado. La comparación se define con mismo protocolo físico y mismo formato de salida, para asegurar una evaluación justa de resultados.

Aspecto	**Trabajos y soluciones previas**	**Enfoque del TFG**
Procesado	Estimación espectral o pipelines cerrados	Pipeline Welch integrado y trazable
Plataforma principal	Boyas dedicadas o prototipos específicos	Android como interfaz principal y núcleo de explotación
Comparación entre nodos	No siempre homogénea	Comparativa Android vs embebido con formato común
Integración extremo a extremo	Frecuentemente fragmentada	Adquisición, procesado, visualización y validación en un mismo sistema

Cuadro 2.1: Posicionamiento resumido del trabajo frente a soluciones previas.

2.8 Posicionamiento del trabajo

La aportación del TFG no reside únicamente en el desarrollo de una aplicación Android, sino en la construcción de un sistema completo de ingeniería que integra:

- adquisición de datos inerciales;
- arquitectura de comunicación entre subsistemas;
- procesado espectral reproducible;

- visualización y explotación de resultados;
- comparación entre dos plataformas de medida.

Por ello, la memoria se orienta a describir un sistema ciberfísico de adquisición y procesado, donde la aplicación móvil actúa como interfaz principal pero no constituye por sí sola el objeto completo del trabajo.

Capítulo 3

Análisis del problema, requisitos, planificación y presupuesto

La ejecución del TFG se organiza como un desarrollo incremental de producto, con validación continua sobre datos reales y simulados.

3.1 Descripción general del sistema a desarrollar

El sistema se plantea como un conjunto ciberfísico en el que intervienen adquisición de datos, comunicación, procesado de señal y visualización. La arquitectura final debe permitir tanto el funcionamiento autónomo del teléfono móvil como la integración de un nodo embebido basado en MCU+ESP32+IMU.

3.2 Requisitos funcionales

Los requisitos funcionales principales del sistema son:

- capturar datos inerciales en modo móvil;
- almacenar sesiones con metadatos de adquisición;
- procesar sesiones mediante un pipeline espectral homogéneo;
- visualizar serie temporal, espectro y métricas derivadas;
- conectarse al ESP32 y gestionar sesiones procedentes del modo embebido;
- comparar resultados entre plataformas con criterios comunes.

3.3 Requisitos no funcionales

Además de las funciones visibles para el usuario, el diseño debe satisfacer criterios de:

- fiabilidad en sesiones largas;
- modularidad entre adquisición, procesado y visualización;
- portabilidad del flujo de trabajo entre plataformas;
- facilidad de uso en campo o en laboratorio;
- bajo coste y viabilidad de despliegue.

3.4 Restricciones técnicas

Las restricciones más relevantes del proyecto son:

- limitaciones de frecuencia efectiva y jitter en Android;
- disponibilidad y comportamiento real de sensores inerciales integrados;
- limitaciones de memoria, conectividad y temporización en ESP32;
- dificultad práctica de integración hardware del IMU externo;
- necesidad de ventanas largas para obtener métricas defendibles.

3.5 Casos de uso principales

Los casos de uso que estructuran el sistema son:

- captura y procesado en modo móvil;
- conexión y consulta de estado del modo ESP32;
- descarga y reprocesado de sesiones externas;
- revisión de resultados y comparación entre plataformas.

Figura pendiente

Diagrama de casos de uso recomendado:
usuario, modo móvil, modo ESP32, procesado, consulta de resultados

Figura 3.1: Casos de uso principales del sistema.

3.6 Enfoque de trabajo

Se adopta una **metodología de desarrollo incremental e iterativa**. La razón de esta elección es que el trabajo combina varias fuentes de incertidumbre: disponibilidad real de sensores, diferencias entre plataformas, integración hardware progresiva y necesidad de validar el procesado antes de cerrar la instrumentación final.

En lugar de plantear un desarrollo lineal cerrado, el proyecto se organizó como una sucesión de iteraciones con entregables funcionales. Cada iteración debía dejar operativo, al menos de forma parcial, uno de los bloques relevantes del sistema y producir evidencia suficiente para decidir el siguiente paso.

De forma resumida, la estrategia de trabajo por bloques funcionales fue la siguiente:

1. Captura IMU y normalización temporal.
2. Procesado espectral on-device.
3. Visualización y exportación de sesión.
4. Modo display para integración con ESP32.
5. Validación comparativa final entre plataformas.

Este enfoque incremental tiene varias ventajas en el contexto del TFG:

- permite validar hipótesis del pipeline antes de depender del hardware definitivo;
- facilita aislar errores de adquisición, de procesado o de integración;
- evita que los problemas de una rama del sistema bloqueen el avance del conjunto;
- produce versiones parciales ya defendibles desde el punto de vista ingenieril.

3.7 Fases del proyecto

- **Fase 1.** Definición de arquitectura del producto y formato común de datos.
- **Fase 2.** Prototipado de captura/procesado con scripts de referencia para reducir riesgo técnico.
- **Fase 3.** Integración on-device en Android y diseño de interfaz operativa.
- **Fase 4.** Integración ESP32+MPU6050 y modo Android display.
- **Fase 5.** Campaña comparativa y elaboración de conclusiones.

3.8 Plan de pruebas y control de riesgos

Los principales riesgos y su mitigación son:

- Ventanas insuficientes para estabilidad espectral: imponer ventana robusta mínima.
- Variabilidad de frecuencia de muestreo: medir f_s real por sesión y registrar jitter.
- Diferencias de formato Android/ESP32: fijar esquema de salida común desde el inicio.
- Deriva o ruido excesivo: incluir validación de calidad y descarte de sesiones no válidas.

3.9 Presupuesto orientativo

El trabajo prioriza bajo coste de hardware:

- teléfono Android: plataforma principal;
- ESP32 con LoRa: disponible en laboratorio personal;
- IMU MPU6050: disponible para fase comparativa.

El coste principal del proyecto se concentra en tiempo de desarrollo, integración y validación experimental.

Capítulo 4

Diseño de la plataforma

El diseño se centra en Android como núcleo del producto, con dos modos operativos y módulos desacoplados de captura, procesado, visualización y exportación.

4.1 Enfoque metodológico de diseño

El diseño del sistema se formula siguiendo un **enfoque modular de sistema ciberfísico**. Este planteamiento resulta adecuado porque el TFG no se limita a un componente software aislado, sino que integra elementos físicos de medida, bloques de comunicación, tratamiento de señal y presentación de resultados.

Desde esta perspectiva, el sistema puede entenderse como la coordinación de cuatro capas:

- una capa de **adquisición**, donde se originan las medidas inerciales;
- una capa de **comunicación**, que transporta datos o resultados entre nodos;
- una capa de **procesado**, que transforma la señal en métricas de estado de mar;
- una capa de **interfaz y visualización**, que hace explotable el resultado.

Este enfoque aporta coherencia arquitectónica al trabajo y justifica que Android no sea descrito simplemente como una aplicación, sino como la interfaz principal de un sistema completo.

4.2 Arquitectura funcional

La arquitectura global se divide en:

- **Modo Android autónomo**: captura y procesa en el propio teléfono.

- **Modo sistema embebido**: adquisición externa con MCU+ESP32+IMU y explotación desde Android.

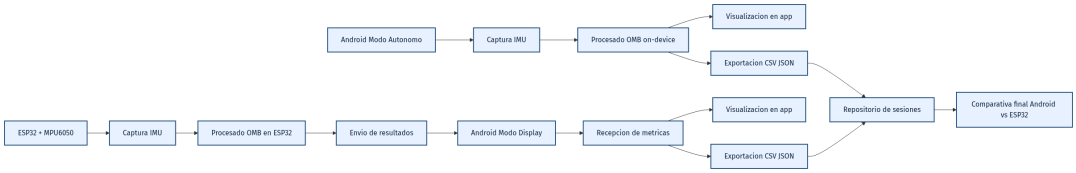

Figura 4.1: Arquitectura funcional del TFG con Android como plataforma central.

4.2.1 Arquitectura física

Desde el punto de vista físico, el sistema se compone de un teléfono Android, una IMU integrada o externa según el modo de operación, un bloque MCU y una placa ESP32 empleada como nodo de comunicación. Las conexiones entre estos elementos determinan el camino de adquisición y condicionan la robustez práctica del sistema.

Figura pendiente

Diagrama físico recomendado:
teléfono Android, ESP32, MCU, IMU, enlaces de comunicación
y alimentación

Figura 4.2: Arquitectura física del sistema y conexiones entre subsistemas.

4.2.2 Arquitectura lógica y software

La arquitectura lógica se organiza en módulos desacoplados: captura, persistencia de sesiones, procesado de señal, comunicación con el subsistema embebido y visualización de resultados. Esta separación reduce acoplamiento y facilita tanto la evolución del producto como la comparativa entre plataformas.

4.3 Diseño del bloque IMU

El bloque IMU se modela como un módulo de producto, no como experimento aislado:

- adquisición de `accelerometer` y `gyroscope`;

- normalización de tiempo y estimación de frecuencia efectiva;
- serialización en formato común para procesado y exportación.

Este diseño permite reutilizar el mismo contrato de datos tanto en Android como en ESP32.

4.4 Diseño del subsistema de adquisición

La adquisición se define como un subsistema independiente del procesado, con el fin de desacoplar la obtención de muestras de las etapas posteriores de almacenamiento, análisis y visualización. Este criterio permite mantener una arquitectura común para ambas plataformas y facilita la comparación experimental.

4.4.1 Adquisición en modo móvil

En el modo móvil, el teléfono se encarga de registrar las magnitudes inerciales disponibles mediante la API de sensores de Android. La captura produce sesiones temporales que posteriormente son normalizadas y procesadas localmente.

4.4.2 Adquisición en modo embebido

En el modo embebido, la adquisición se apoya en una IMU externa y un subsistema MCU+ESP32. La función del ESP32 es actuar como elemento de enlace con la aplicación Android, bien como nodo de diagnóstico, bien como pasarela de datos y resultados.

4.4.3 Diseño del almacenamiento de sesiones

Las sesiones de captura se almacenan con formato homogéneo y metadatos explícitos, permitiendo:

- persistir los datos en bruto;
- reprocesar sesiones con distintos parámetros;
- comparar resultados entre plataformas de forma reproducible.

4.4.4 Ventanas temporales y duración de captura

El diseño de adquisición contempla explícitamente la duración de las sesiones como variable crítica del sistema. Las ventanas largas se reservan para resultados cuantitativos robustos, mientras que las ventanas cortas se interpretan como mecanismo de inspección rápida y diagnóstico operacional.

4.5 Diseño del flujo Android

Cada sesión incluye:

- series temporales inerciales;
- metadatos de dispositivo y parámetros de cálculo;
- métricas finales (H_s, T_z, T_c, f_s real, indicadores de calidad).

4.6 Flujo de datos del sistema

El flujo de datos completo va desde el sensor hasta la obtención de parámetros estimados. Primero se adquieren muestras y marcas temporales, después se normalizan y almacenan, a continuación se aplica el bloque espectral y finalmente se representan métricas y gráficas en la aplicación. Este flujo extremo a extremo es uno de los elementos que distinguen el trabajo como sistema de ingeniería completo.

4.7 Diseño de comparativa entre plataformas

La comparativa se diseña para medir diferencias de estabilidad, ruido, latencia y consumo bajo un mismo protocolo de prueba.

4.7.1 Modo móvil

En este modo el teléfono concentra adquisición, procesado y visualización. Su principal ventaja es la simplicidad operativa y la disponibilidad inmediata; su principal limitación es el menor control sobre el sensor y la electrónica de medida.

4.7.2 Modo sistema embebido

En este modo el nodo externo asume la adquisición física y Android actúa como interfaz de supervisión, descarga y análisis. La ventaja principal es la modularidad del sistema; la principal dificultad reside en la integración hardware y en la estabilidad real del camino de adquisición.

4.8 Diseño del subsistema de procesado

El procesado se concibe como un bloque independiente que transforma datos inerciales en parámetros estadísticos del estado del mar. Su diseño debe ser lo bastante modular para

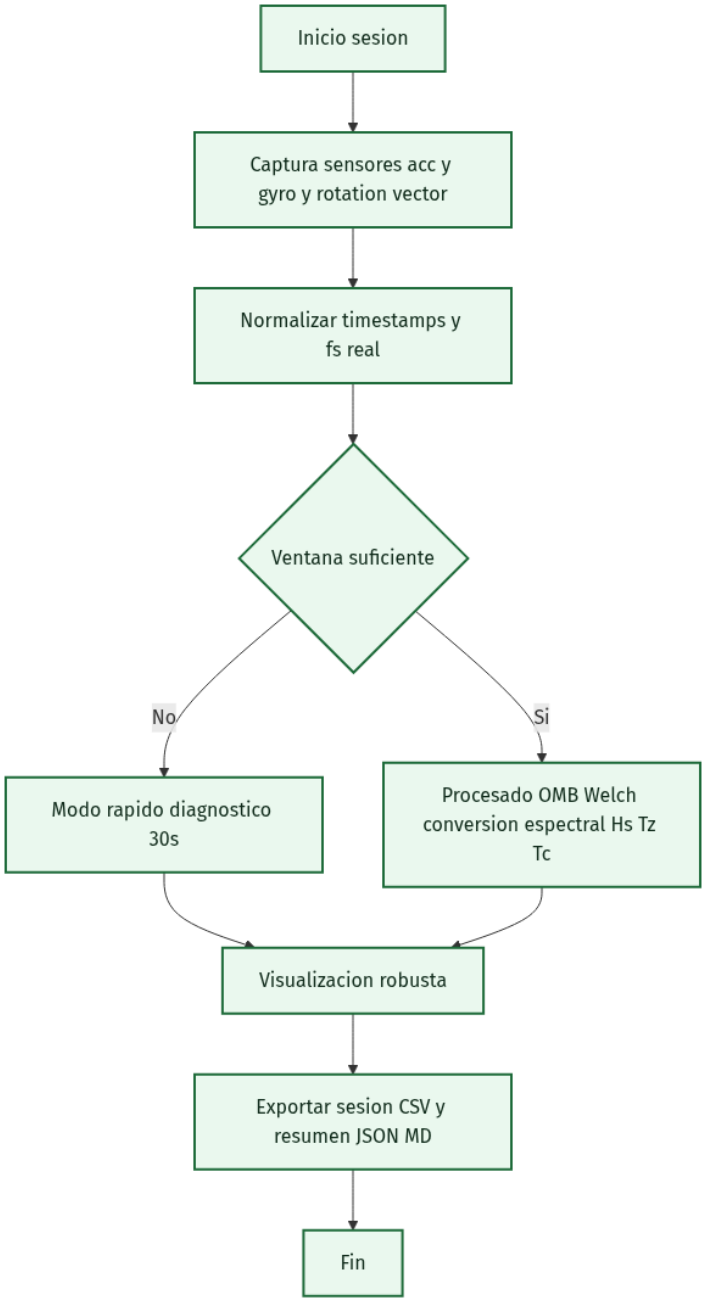

Figura 4.3: Flujo Android: captura, procesado on-device, visualización y exportación.

Figura pendiente

Pipeline recomendado:
sensor → sesión → preprocesado → Welch
→ momentos espectrales → métricas → visualización

Figura 4.4: Flujo de datos extremo a extremo del sistema.

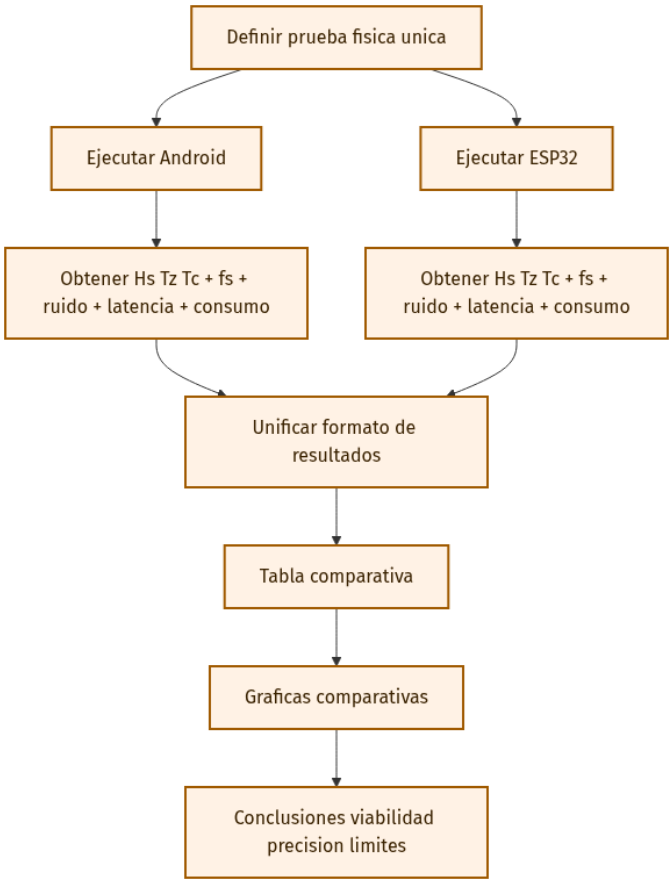

Figura 4.5: Flujo de comparativa entre plataformas con métricas unificadas.

ejecutarse tanto sobre datos capturados por el teléfono como sobre sesiones procedentes del sistema embebido.

4.8.1 Metodología de procesado espectral

La metodología de tratamiento de señal adoptada en este TFG es una **metodología de procesado espectral basada en Welch**. No se emplea únicamente como una técnica numérica puntual, sino como el marco metodológico principal para pasar de una señal inercial ruidosa y dependiente del sensor a parámetros estadísticos comparables entre plataformas.

El motivo de esta elección es doble. Por una parte, el oleaje se caracteriza de forma natural mediante descriptores estadísticos obtenidos sobre ventanas temporales suficientemente largas. Por otra, el análisis espectral es más robusto que una doble integración temporal directa cuando se trabaja con medidas afectadas por ruido, deriva y variabilidad de frecuencia de muestreo.

4.8.2 Entrada y preprocesado

Las señales de entrada incluyen aceleración y velocidad angular junto con sus marcas temporales. Antes del cálculo espectral se realiza uniformización temporal, estimación de frecuencia efectiva de muestreo y obtención de las magnitudes dinámicas relevantes.

Desde el punto de vista del diseño, el preprocesado cumple una función crítica: reducir la dependencia del resultado final respecto a irregularidades de adquisición, diferencias entre plataformas y ruido no informativo. Por ello, no se concibe como una etapa auxiliar, sino como una parte estructural del subsistema de procesado.

Las operaciones de preprocesado consideradas son:

- validación básica de la sesión y descarte de capturas manifiestamente inválidas;
- ordenación temporal y cálculo de Δt entre muestras;
- estimación de la frecuencia efectiva f_s y del jitter temporal;
- selección de la componente o magnitud inercial más útil para representar el movimiento vertical;
- separación entre contribución gravitatoria y componente dinámica;
- preparación de una serie temporal apta para análisis espectral.

Este enfoque persigue que el mismo pipeline pueda operar sobre datos procedentes de Android y del sistema embebido sin introducir ramificaciones algorítmicas excesivamente distintas.

Figura pendiente

Diagrama de preprocesado recomendado:
timestamps, f_s real, jitter, selección de canal,
señal dinámica útil

Figura 4.6: Bloque de entrada y preprocesado del pipeline.

4.8.3 Bloque espectral

El bloque espectral implementa un estimador tipo Welch y la obtención de momentos espectrales. Este bloque constituye el núcleo analítico del sistema, al producir las métricas principales H_s, T_z, T_p y T_c.

La elección de Welch responde a un compromiso ingenieril entre tres necesidades del proyecto: robustez frente a ruido y variabilidad temporal, coste computacional razonable para ejecución *on-device* y trazabilidad conceptual del resultado. Frente a métodos temporales más directos, la estimación espectral permite explotar mejor el carácter estadístico del problema y desacoplar la solución de la reconstrucción instantánea de altura.

El diseño del bloque espectral contempla:

- segmentación de la señal en ventanas parciales;
- aplicación de una función ventana para reducir fuga espectral;
- solapamiento entre segmentos consecutivos;
- promedio de densidades espectrales parciales;
- cálculo de momentos sobre el espectro resultante.

La salida intermedia de esta etapa no es únicamente un conjunto de métricas, sino también una representación física interpretable del contenido energético de la señal a distintas frecuencias.

4.8.4 Cálculo de momentos espectrales

Los momentos espectrales proporcionan un puente directo entre la PSD estimada y los parámetros de estado de mar. Si $S_\eta(f)$ representa el espectro de elevación, los momentos se obtienen mediante:

$$m_n = \int f^n S_\eta(f)\, df$$

En el contexto de este trabajo, los momentos más relevantes son m_0, m_2 y m_4, ya que permiten construir métricas comparables entre plataformas con una formulación compacta y trazable.

4.8.5 Obtención de parámetros de oleaje

A partir de los momentos y del espectro estimado, el sistema calcula:

- H_s como estimador de altura significativa, ligado a la energía total contenida en la señal;
- T_z como descriptor temporal medio derivado de la relación entre momentos;
- T_p como período asociado al máximo espectral dominante;
- T_c como período característico adicional sensible a la forma global del espectro.

Estos parámetros forman el núcleo del resultado cuantitativo del sistema y, por tanto, del diseño experimental posterior.

Figura pendiente

Diagrama recomendado:
PSD $\rightarrow$ momentos $m_0, m_2, m_4 \rightarrow H_s, T_z, T_p, T_c$

Figura 4.7: Conversión desde espectro de elevación a parámetros de oleaje.

4.8.6 Interpretación física de resultados

Las salidas del procesado se interpretan como descriptores estadísticos del estado del mar. Este matiz es esencial para evitar una lectura incorrecta del sistema como si midiese altura instantánea de una ola individual con precisión absoluta.

En consecuencia, el objetivo del diseño no es producir una trayectoria vertical absoluta libre de deriva, sino una estimación suficientemente robusta de parámetros agregados sobre una ventana temporal. Esta decisión es coherente con la literatura técnica, con la naturaleza del sensado inercial y con el tipo de validación que puede defenderse en el marco de este TFG.

4.8.7 Salida de resultados

La salida del procesado incluye:

- parámetros finales de oleaje;
- indicadores de calidad o validez de la ventana;
- representación gráfica de serie temporal y espectro.

El diseño de salida también contempla que el sistema conserve información suficiente para auditoría técnica: frecuencia efectiva, trazas intermedias, parámetros de ejecución y, cuando proceda, advertencias sobre limitaciones de la sesión analizada.

4.9 Diseño hardware del sistema embebido

Desde el punto de vista físico, el sistema embebido se compone de una placa ESP32, una IMU y la electrónica auxiliar necesaria para adquisición y comunicación. La memoria debe tratar este bloque como parte de la arquitectura global del sistema, incluyendo tanto su diseño funcional como las dificultades reales de integración detectadas durante el desarrollo.

Figura pendiente

Esquema hardware recomendado:
ESP32, MCU, IMU, OLED y pinout de conexiones relevantes

Figura 4.8: Esquema del subsistema embebido y conexiones principales.

Capítulo 5

Implementación del sistema

En este capítulo se describe la implementación técnica del producto y las decisiones adoptadas para garantizar reproducibilidad.

5.1 Captura y normalización

La captura en Android registra aceleración y velocidad angular con timestamp. La normalización posterior calcula la frecuencia efectiva de muestreo y corrige irregularidades temporales.

5.1.1 Implementación de la captura en Android

La aplicación Android implementa un modo de captura local basado en los sensores integrados del dispositivo. Esta implementación constituye el primer camino funcional completo del sistema y sirve como referencia para el diseño del resto de subsistemas.

5.1.2 Implementación del formato de sesión

Cada sesión se persiste con un formato uniforme, pensado para soportar reprocesado, exportación y comparación entre dispositivos. Además de los datos crudos, se incluyen metadatos relevantes como duración, frecuencia efectiva o modo de adquisición.

5.1.3 Decisiones de diseño del almacenamiento

El formato de sesión se diseñó para priorizar trazabilidad y reutilización. Esto permite usar la misma sesión para depuración, validación experimental y visualización, evitando bifurcar formatos específicos para cada plataforma.

5.2 Pipeline de procesado integrado

El método implementado sigue una secuencia espectral tipo OpenMetBuoy:

1. estimación de f_s real;
2. separación de gravedad para obtener aceleración dinámica vertical;
3. PSD de aceleración mediante Welch;
4. conversión a elevación:
$$S_\eta(f) = \frac{S_a(f)}{(2\pi f)^4}$$
5. cálculo de momentos y métricas:
$$H_s = 4\sqrt{m_0}, \qquad T_z = \frac{1}{\sqrt{m_2/m_0}}, \qquad T_c = \frac{1}{\sqrt{m_4/m_2}}$$

Se trata de un método adoptado e integrado en la plataforma, no de una propuesta algorítmica original [3].

En este capítulo, la implementación debe leerse como la concreción práctica de la metodología de procesado descrita en el diseño. Es decir, no se trata solo de una secuencia de funciones, sino de la materialización software de un enfoque espectral elegido por su coherencia con el dominio del problema y con las limitaciones reales del sistema.

La implementación práctica del pipeline no consiste únicamente en traducir fórmulas a código. En un sistema real, el rendimiento del método depende de cómo se trate la irregularidad temporal, de la forma en que se estabilice la señal de entrada y de que la salida final conserve trazabilidad suficiente para poder justificar cada valor obtenido. Por ello, la implementación se diseñó como una cadena de etapas claramente separadas, cada una con entradas, salidas y magnitudes de control verificables.

5.2.1 Estimación de frecuencia efectiva y calidad temporal

La primera operación consiste en estimar la frecuencia efectiva real de adquisición a partir de los *timestamps* registrados. Esta decisión es importante porque, en la práctica, ni Android ni un sistema embebido genérico garantizan exactamente la frecuencia teórica solicitada.

El pipeline calcula:

- la duración efectiva de la sesión;
- el espaciado medio entre muestras;

- la frecuencia efectiva f_s;
- indicadores simples de jitter o irregularidad temporal.

Estas magnitudes no son accesorias: afectan directamente a la interpretación del eje de frecuencias y, por tanto, al valor final de los parámetros derivados.

5.2.2 Obtención de la señal dinámica útil

Una vez caracterizada la base temporal, la implementación selecciona y transforma las señales inerciales para aproximarse a la componente dinámica de interés. En particular, se busca aislar la aceleración útil frente a la contribución gravitatoria y frente a cambios lentos que podrían contaminar el análisis espectral.

Esta etapa constituye uno de los puntos más delicados del pipeline, ya que una mala separación entre señal útil y componentes espurias puede degradar el espectro y falsear de manera sistemática los momentos calculados.

Figura pendiente

Figura recomendada:
comparativa entre señal cruda y señal dinámica preprocesada

Figura 5.1: Ejemplo de señal inercial antes y después del preprocesado.

5.2.3 Estimación de PSD mediante Welch

La implementación de Welch se apoya en segmentación, solapamiento y promediado espectral. Su objetivo no es únicamente producir una PSD suave, sino hacerlo con un coste asumible para el contexto del trabajo: ejecución *on-device*, necesidad de repetición sobre múltiples sesiones y comparabilidad entre plataformas con distinta calidad de adquisición.

En términos operativos, el bloque genera una estimación $S_a(f)$ para la aceleración y posteriormente deriva el espectro de elevación:

$$S_\eta(f) = \frac{S_a(f)}{(2\pi f)^4}$$

Este paso merece una interpretación cuidadosa. La división por $(2\pi f)^4$ traduce el problema desde el dominio de la aceleración al dominio de la elevación bajo el marco lineal adoptado.

Sin embargo, también amplifica la sensibilidad a bajas frecuencias, por lo que la estabilidad del preprocesado y la elección de bandas válidas resultan críticas.

Figura pendiente

Figura recomendada:
PSD de aceleración y espectro de elevación derivados
en una misma sesión

Figura 5.2: Ejemplo de estimación espectral y conversión a elevación.

5.2.4 Implementación de modos de procesado

La implementación contempla distintos modos de procesado según la frecuencia efectiva y el contexto de captura. Esto permite mantener coherencia entre datos procedentes del teléfono móvil y sesiones generadas o transferidas desde el sistema embebido.

En la práctica, esta distinción permite separar:

- un modo orientado a resultados robustos sobre ventanas largas;
- un modo más rápido o exploratorio para diagnóstico de sesiones cortas;
- un modo de análisis equivalente para datos descargados desde el subsistema embebido.

De este modo, el sistema no fuerza una única configuración rígida, pero mantiene un contrato común sobre qué métricas son comparables y bajo qué condiciones pueden considerarse válidas.

5.2.5 Cálculo de momentos espectrales

La implementación obtiene momentos como m_0, m_2 y m_4 a partir del espectro estimado, y los reutiliza como base común para el cálculo de métricas de oleaje. Este diseño evita introducir fórmulas desconectadas entre sí y facilita verificar la coherencia del pipeline completo.

Concretamente, la implementación utiliza:

$$H_s = 4\sqrt{m_0}, \qquad T_z = \frac{1}{\sqrt{m_2/m_0}}, \qquad T_c = \frac{1}{\sqrt{m_4/m_2}}$$

Además, T_p se obtiene a partir de la frecuencia correspondiente al máximo del espectro de elevación. La ventaja de esta formulación es que todas las métricas quedan ancladas a una

misma representación espectral, lo que simplifica la validación cruzada y evita incoherencias internas del sistema.

5.2.6 Trazabilidad del resultado

Una decisión importante de implementación ha sido conservar no solo las métricas finales, sino también señales y resultados intermedios cuando el flujo de trabajo lo requiere. Esto incluye frecuencia efectiva, espectro estimado, serie temporal preprocesada y parámetros de configuración utilizados en el cálculo.

Esta trazabilidad tiene valor ingenieril directo: permite depurar, comparar sesiones, justificar discrepancias entre plataformas y explicar por qué un resultado concreto debe considerarse robusto, dudoso o no válido.

5.3 Visualización y exportación

La aplicación muestra:

- evolución temporal de olas $h(t)$;
- panel de métricas actuales;
- indicador de validez de ventana para resultado robusto.

La exportación de sesión incluye datos crudos, datos procesados, métricas y parámetros de ejecución.

5.4 Implementación de la aplicación Android

La aplicación se implementa como interfaz principal del sistema, integrando captura, gestión de sesiones, procesado y visualización. Su función es coordinar el flujo extremo a extremo y presentar resultados homogéneos independientemente del origen de la captura.

5.4.1 Modo móvil

El modo móvil permite capturar datos con los sensores del teléfono, almacenarlos y procesarlos localmente. Representa la versión autónoma y autosuficiente del sistema.

5.4.2 Modo ESP32

El modo ESP32 se implementa como vía de conexión con el subsistema embebido. Incluye funciones de verificación de conexión, gestión de capturas, descarga de sesiones y presentación de resultados de forma equivalente al modo móvil.

5.4.3 Gestión de sesiones y visualización

La aplicación incluye mecanismos de nombrado, selección, borrado y reprocesado de sesiones, así como vistas para espectro, serie temporal y métricas resumidas.

5.4.4 Estructura funcional de la app

La aplicación se organiza en torno a flujos diferenciados de captura, consulta y procesado. Esta estructura permite extender el sistema sin convertir la interfaz en una colección de pantallas inconexas y facilita mantener un comportamiento comparable entre modo móvil y modo ESP32.

Figura pendiente

Diagrama recomendado:
navegación principal de la app y relación entre pantallas

Figura 5.3: Flujo de navegación de la aplicación Android.

5.4.5 Decisiones de diseño software

La separación entre adquisición, almacenamiento, procesado y visualización responde a un criterio de mantenibilidad. Este enfoque también facilita depuración, introducción de nuevos modos y reutilización de componentes en fases posteriores del proyecto.

5.5 Implementación de referencia para iteración técnica

Durante el desarrollo se empleó un entorno de scripts Python para acelerar pruebas de parámetros y depuración del pipeline. Esta capa de soporte queda documentada como herramienta de ingeniería, mientras que el producto objetivo es la ejecución en Android.

La base de captura de sensores en Android se apoyó en la documentación oficial [4].

5.6 Implementación del subsistema embebido

La implementación embebida se ha centrado en el ESP32 como nodo de comunicación y experimentación, incorporando funciones de red, diagnóstico y simulación de capturas. Ade-

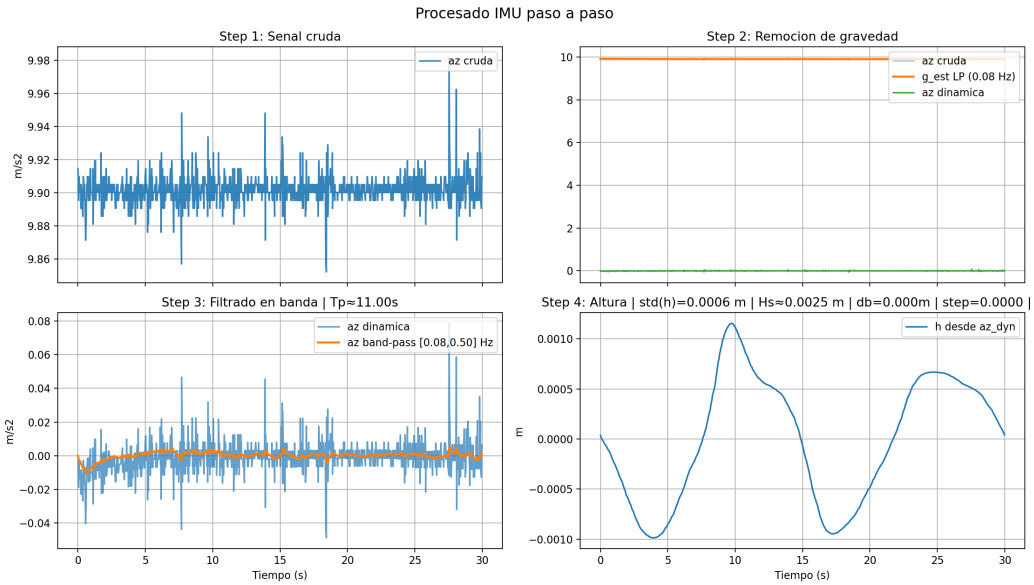

Figura 5.4: Visualización por etapas del pipeline utilizada en la fase de validación.

más, se desarrollaron herramientas específicas de diagnóstico para validar la integración del IMU y acotar problemas de bus, alimentación o cableado.

5.6.1 Firmware principal del ESP32

El firmware principal ofrece conectividad, mecanismos de intercambio con la aplicación Android y elementos de soporte al desarrollo, como la pantalla OLED y trazas de estado.

5.6.2 Firmware y scripts de diagnóstico

Durante el desarrollo se implementaron utilidades específicas para diagnóstico del bus I^2C y validación del IMU, tanto en C/ESP-IDF como en MicroPython. Estas herramientas se consideran parte del trabajo de ingeniería realizado para asegurar la trazabilidad de los problemas de integración hardware.

5.6.3 Estado actual y evolución prevista

En el estado actual del proyecto, el ESP32 ya funciona como nodo de comunicación, diagnóstico y simulación de sesiones. La línea de evolución pendiente consiste en consolidar la integración física del IMU/MCU, estabilizar el camino de adquisición externo y cerrar la comparativa experimental con el mismo pipeline analítico usado en Android.

Figura pendiente

Figura recomendada:
captura del modo ESP32, OLED del dispositivo o monitor serie

Figura 5.5: Estado actual del subsistema embebido durante desarrollo y diagnóstico.

Capítulo 6

Validación y evaluación

La validación se enfoca en comprobar la viabilidad del producto en condiciones de uso real y en preparar una comparación defendible entre plataformas.

6.1 Metodología experimental

La evaluación se diseña con enfoque de ingeniería aplicada, es decir, no solo para comprobar que el sistema produce salidas, sino para estudiar bajo qué condiciones dichas salidas son estables, comparables y útiles. Esto obliga a distinguir claramente entre pruebas funcionales, pruebas de procesado y pruebas de comparación entre plataformas.

En consecuencia, la validación sigue una **metodología experimental comparativa**. Su objetivo no es únicamente verificar que Android y el sistema embebido producen algún resultado, sino comprobar si dichos resultados pueden contrastarse de forma técnica y defendible bajo condiciones de adquisición homogéneas.

Esta metodología comparativa se apoya en tres principios:

- **mismo protocolo de observación**: condiciones físicas y ventanas temporales equivalentes;
- **mismo marco analítico**: uso de métricas y pipeline de procesado compatibles;
- **misma lógica de interpretación**: evaluación tanto cuantitativa como cualitativa de la robustez del sistema.

6.1.1 Escenarios de prueba

Los escenarios de prueba contemplan sesiones en modo móvil, sesiones transferidas o simuladas desde el sistema embebido y pruebas específicas de integración hardware. Esta separación permite analizar tanto el comportamiento algorítmico como la robustez real del sistema completo.

Figura pendiente

Diagrama recomendado:
protocolo experimental, escenarios y flujo de comparación

Figura 6.1: Protocolo experimental seguido en la validación.

6.2 Estrategia temporal de evaluación

La validación distingue dos escenarios:

- **Ventana robusta (10–20 min)**: cálculo principal para métricas finales.
- **Ventana corta (∼30 s)**: diagnóstico rápido de señal y funcionamiento.

Los resultados robustos del TFG se reportan sobre ventana larga.

6.3 Configuración de las pruebas

Para cada campaña experimental deben documentarse explícitamente:

- plataforma utilizada (Android o sistema embebido);
- duración de captura;
- frecuencia efectiva de muestreo;
- condiciones físicas de la prueba;
- versión del software o firmware empleado.

Esta información resulta crítica para reproducibilidad y para interpretación correcta de los resultados.

6.3.1 Procedimiento de análisis

Para cada sesión se sigue un procedimiento común: revisión de metadatos, cálculo de frecuencia efectiva, procesado espectral, extracción de parámetros de oleaje y almacenamiento de métricas finales. Solo a partir de ese flujo homogéneo puede realizarse una comparación defendible entre plataformas.

6.4 Métricas de evaluación

Las métricas principales son:

- H_s, T_z, T_c y su estabilidad temporal;
- T_p como descriptor del pico dominante;
- frecuencia efectiva de muestreo y jitter;
- latencia de cálculo y visualización;
- consumo por sesión;
- indicadores de calidad de señal.

6.5 Plantilla de comparativa Android vs ESP32+MPU6050

Métrica	Android	ESP32+MPU6050	Criterio de evaluación
Frecuencia efectiva (Hz)	Medida en sesión	Medida en sesión	Estabilidad temporal y jitter
H_s, T_z, T_c	Serie temporal	Serie temporal	Concordancia entre plataformas
Ruido residual	Estimado tras procesado	Estimado tras procesado	Robustez de la estimación
Latencia	On-device / display	Envío + display	Respuesta en uso real
Consumo energético	Por sesión de prueba	Por sesión de prueba	Viabilidad operativa

Cuadro 6.1: Plantilla de comparación Android vs ESP32+MPU6050 con protocolo homogéneo.

6.6 Criterios de aceptación

Se considerará la plataforma validada cuando:

- complete el flujo captura-procesado-visualización-exportación sin intervención externa;
- mantenga estabilidad de métricas en sesiones largas repetidas;
- documente límites de precisión y condiciones de no validez;
- permita comparar Android y ESP32 con resultados trazables.

6.7 Resultados

Este apartado se reserva para presentar los resultados de forma estructurada, separando claramente la procedencia de los datos y el tipo de análisis realizado.

6.7.1 Resultados del modo móvil

Aquí deben incluirse los parámetros obtenidos en sesiones capturadas íntegramente con el teléfono móvil, junto con ejemplos representativos de espectro, serie temporal y estabilidad temporal de las métricas.

Figura pendiente

Resultado pendiente:
captura real del modo móvil con serie temporal y métricas

Figura 6.2: Resultados representativos de una sesión en modo móvil.

6.7.2 Resultados del sistema embebido

Este bloque debe documentar los resultados disponibles del modo embebido, incluyendo tanto sesiones válidas como limitaciones encontradas durante la integración de hardware cuando afecten a la interpretación de los datos.

Figura pendiente

Resultado pendiente:
sesión procedente del sistema embebido o captura del modo ESP32

Figura 6.3: Resultados representativos del modo embebido.

6.7.3 Resultados del procesado espectral

En esta subsección conviene mostrar ejemplos donde pueda observarse la relación entre señal de entrada, PSD estimada y parámetros derivados. Es una pieza importante para defender que el resultado no es una cifra aislada, sino la salida de un pipeline trazable.

Para que esta parte sea realmente sólida en la memoria, conviene presentar al menos:

- una serie temporal representativa de entrada;
- la estimación espectral correspondiente;
- los momentos relevantes calculados;
- la traducción final a H_s, T_z, T_p y T_c.

Este encadenamiento ayuda a demostrar que el sistema no actúa como una "caja negra", sino como una implementación verificable de un método espectral bien definido.

Figura pendiente

Figura recomendada:
serie temporal, PSD y parámetros finales de una misma sesión

Figura 6.4: Ejemplo completo del procesado espectral sobre una sesión.

6.7.4 Comparación móvil vs sistema embebido

La comparación debe presentarse con criterios cuantitativos y cualitativos: coherencia entre parámetros, estabilidad de muestreo, facilidad de operación y robustez de adquisición.

6.7.5 Resultados de robustez e integración

Además de los resultados numéricos, este apartado debe recoger incidencias de adquisición, estabilidad del bus, problemas de integración y su impacto sobre la validez experimental del sistema.

Figura pendiente

Figura recomendada:
tabla o gráfico comparativo entre Android y sistema embebido

Figura 6.5: Comparación de resultados entre plataforma móvil y sistema embebido.

6.8 Discusión de resultados

Además de reportar valores numéricos, la evaluación debe incluir una discusión crítica sobre:

- estabilidad de los parámetros espectrales;
- sensibilidad a duración de ventana y frecuencia de muestreo;
- diferencias observadas entre plataforma móvil y embebida;
- impacto de las dificultades de integración hardware en la validez de las conclusiones.

Esta discusión es especialmente relevante en un TFG de ingeniería, ya que pone de manifiesto tanto el comportamiento del sistema como el criterio técnico aplicado durante el desarrollo.

6.8.1 Análisis crítico de los resultados

Aquí debe interpretarse qué resultados son sólidos, cuáles son provisionales y qué decisiones de diseño han influido en ellos.

En particular, el análisis crítico debe relacionar estabilidad de resultados con:

- duración de la ventana;
- calidad temporal de la adquisición;
- diferencias de hardware entre plataformas;
- robustez del preprocesado frente a ruido y deriva.

6.8.2 Validez del enfoque basado en IMU

Esta subsección debe remarcar qué magnitudes pueden estimarse con fiabilidad razonable y qué limitaciones son inherentes al uso de sensores inerciales para caracterizar el estado del mar.

6.8.3 Ventajas e inconvenientes de cada plataforma

La comparación entre móvil y sistema embebido debe ir más allá de las métricas finales e incluir facilidad de despliegue, control del hardware, reproducibilidad y esfuerzo de integración.

6.8.4 Limitaciones del trabajo

En este punto deben recogerse limitaciones de hardware, alcance experimental, validación disponible y tiempo de desarrollo.

6.8.5 Lecciones aprendidas

Este cierre de discusión permite destacar criterio de ingeniería, especialmente en lo relativo a integración extremo a extremo, depuración hardware y separación entre hipótesis del sistema y restricciones observadas en la práctica.

Capítulo 7

Conclusiones

El trabajo queda reorientado y consolidado en torno a una meta clara: demostrar que un teléfono móvil puede actuar como plataforma ubicua para captura y procesado inercial, con una metodología reproducible y extensible a otras plataformas.

7.1 Conclusiones principales

- El enfoque de plataforma Android como núcleo funcional es viable y coherente con los objetivos del TFG.
- El método espectral tipo OpenMetBuoy proporciona una base técnica sólida para estimar H_s, T_z y T_c.
- La estabilidad del método depende críticamente de la ventana temporal: las ventanas largas (10–20 min) son el modo principal de estimación robusta.
- El modo rápido en ventanas cortas es útil para diagnóstico operativo, pero no debe considerarse equivalente al cálculo robusto.
- La definición de un flujo común Android/ESP32 permite una comparativa final técnicamente defendible.

7.2 Conclusión metodológica clave (a remarcar en la defensa)

- El pipeline adoptado (Welch + momentos espectrales m_0, m_2, m_4) debe presentarse como un estimador de **estado de mar por ventana**, no como estimador de **altura instantánea absoluta** muestra a muestra.
- La razón técnica es estructural: obtener altura instantánea desde IMU implica doble integración de aceleración, muy sensible a deriva, orientación y ruido acumulado.

- Por ello, los resultados principales del TFG son H_s, T_z, T_c y T_p sobre ventanas temporales, y no una "verdad de referencia" en cada instante.
- La señal temporal $h(t)$ se mantiene como apoyo visual y diagnóstico de dinámica, pero no como magnitud primaria de validación cuantitativa.

Esta conclusión no es una limitación accidental de implementación, sino una propiedad del enfoque inercial empleado y está alineada con la literatura revisada (OpenMetBuoy, comparativas smartphone–boya y trabajos de validación operacional).

7.3 Implicaciones prácticas para el diseño experimental

- Sí es válido dividir una captura larga en subventanas para observar evolución temporal, pero con compromiso entre resolución temporal y robustez estadística.
- Subventanas muy cortas (por ejemplo, 1–2 min) pueden usarse para tendencia, aunque con mayor varianza y menor fiabilidad que una ventana de referencia larga.
- Aumentar la frecuencia de muestreo mejora la representación de altas frecuencias y reduce aliasing, pero **no** elimina la necesidad de una duración mínima de ventana para estimar métricas espectrales estables.
- La comparativa Android vs ESP32 debe realizarse con mismo protocolo físico, mismo algoritmo y mismas ventanas para que la conclusión sea defendible.

7.4 Aportaciones

Las aportaciones del trabajo en su estado actual son:

- Definición de arquitectura dual en Android (modo autónomo y modo display).
- Formalización de pipeline de procesado y métricas de salida unificadas.
- Esquemas funcionales y plantilla de evaluación comparativa entre plataformas.
- Entorno reproducible para validación previa de decisiones de diseño.
- Integración de una visión de sistema completo que combina adquisición, comunicación, procesado y visualización.

7.5 Trabajo futuro

Las siguientes líneas continúan directamente el flujo definido:

- Implementar el procesado on-device completo en Android como camino principal.
- Implementar el flujo equivalente en ESP32+MPU6050 con mismo formato de resultados.
- Ejecutar campaña de pruebas comparativas con protocolo físico común.
- Incorporar estimación de calidad de señal y criterios automáticos de validez por sesión.
- Automatizar exportación de informe por sesión para análisis y memoria final.

7.5.1 Líneas futuras prioritarias

Las líneas futuras con mayor impacto académico y técnico son la validación comparativa en condiciones físicas homogéneas, la consolidación del sistema embebido y la mejora de la trazabilidad experimental mediante campañas más largas y repetibles.

Apéndices

Apéndice A

Material adicional

En este apéndice se recoge material de apoyo para reproducir el entorno experimental y el flujo de trabajo técnico del proyecto.

A.1 Scripts principales

- `capture_http_imu.py`: recepción HTTP y guardado de sesiones.
- `process_imu_session.py`: procesado offline con métricas y visualización.
- `process_imu_step_by_step.py`: visualización por etapas del pipeline.

A.2 Comandos de referencia

A.2.1 Captura

```
python3 capture_http_imu.py --host 0.0.0.0 --port 8001 --seconds 30
```

A.2.2 Procesado rápido sin gráfica

```
python3 process_imu_session.py session_YYYYMMDD_HHMMSS_30s.csv
    --no-plot
```

A.2.3 Procesado paso a paso con validación temporal

```
python3 process_imu_step_by_step.py session_YYYYMMDD_HHMMSS_30s.csv
    \
  --mode step4 --g-fc 0.03 --fmin 0.02 --fmax 1.20 \
  --height-min-change 0.00005 --height-step-min 0.000001
    --confirm-samples 6
```

A.3 Datos de prueba

Se trabajó con múltiples sesiones de 30 segundos en formato CSV, incluyendo capturas de reposo y capturas con movimiento vertical manual. La bitácora de resultados y ajustes se mantiene en `INFORME_IMU.md`.

Bibliografía

[1] D. H. Titterton and J. L. Weston, *Strapdown Inertial Navigation Technology*, 2nd ed. The Institution of Engineering and Technology, 2004.

[2] J. A. Farrell, *Aided Navigation: GPS with High Rate Sensors.* McGraw-Hill, 2008.

[3] A. V. Oppenheim and R. W. Schafer, *Discrete-Time Signal Processing*, 3rd ed. Prentice Hall, 2010.

[4] Android Developers, "Sensor overview," https://developer.android.com/guide/topics/sensors/sensors_overview, 2026, accedido el 18 de febrero de 2026.